

RAPPORT DE PROJET

Phase 4 — Tests de Sécurité

Projet : Secure API — Cybersécurité

Date : Mai 2026

1. Introduction

La Phase 4 constitue l'étape finale du projet Secure API. Après avoir construit l'API (Phase 1), implémenté l'authentification (Phase 2) et démontré des failles volontaires (Phase 3), on utilise maintenant des outils professionnels de cybersécurité pour tester et valider la sécurité de l'application.

Cette phase simule exactement ce que fait un pentester (testeur d'intrusion) en entreprise : utiliser des outils spécialisés pour scanner les ports ouverts, intercepter les requêtes HTTP et détecter automatiquement les vulnérabilités.

1.1 Objectifs de la Phase 4

- Scanner les ports ouverts du serveur avec Nmap
- Intercepter et modifier des requêtes HTTP avec Burp Suite
- Démontrer une attaque SQL Injection depuis Burp Suite
- Lancer un scan automatique de vulnérabilités avec OWASP ZAP
- Analyser et interpréter les résultats des scans

2. Définitions des concepts clés

2.1 Concepts généraux

Pentest (Test d'intrusion) : Processus qui consiste à attaquer volontairement une application pour en découvrir les failles de sécurité avant qu'un vrai attaquant ne le fasse. Le pentester utilise les mêmes outils et techniques qu'un attaquant, mais dans un cadre légal et autorisé.

Reconnaissance : Première étape de toute attaque — collecter des informations sur la cible. Cela inclut scanner les ports ouverts, identifier les services qui tournent, détecter les technologies utilisées. C'est exactement ce que fait Nmap.

Port : Numéro entre 0 et 65535 qui identifie un service spécifique sur un serveur. Chaque service utilise un port précis : port 80 pour HTTP, port 443 pour HTTPS, port 3000 pour notre API, port 3306 pour MySQL. Scanner les ports permet de savoir quels services sont exposés.

Firewall (Pare-feu) : Système de sécurité qui filtre le trafic réseau en autorisant ou bloquant les connexions selon des règles définies. En production, on fermerait tous les ports inutiles avec un firewall pour réduire la surface d'attaque.

Header HTTP (En-tête HTTP) : Information supplémentaire envoyée avec chaque requête ou réponse HTTP. Certains headers révèlent des informations sensibles sur le serveur (comme X-

Powered-By: Express), d'autres servent à sécuriser l'application (comme Content-Security-Policy).

Surface d'attaque : L'ensemble des points d'entrée potentiels qu'un attaquant peut exploiter pour compromettre un système. Plus il y a de ports ouverts, de services exposés et d'endpoints non sécurisés, plus la surface d'attaque est grande.

2.2 Nmap

Nmap (Network Mapper) : Outil open source de scan réseau utilisé pour découvrir les hôtes et services sur un réseau. C'est l'un des outils les plus utilisés en cybersécurité, aussi bien par les pentesters que par les administrateurs réseau pour auditer leur infrastructure.

Scan de ports : Technique qui consiste à envoyer des paquets réseau à un serveur sur différents ports pour déterminer lesquels sont ouverts et quels services y tournent. Un port ouvert signifie qu'un service écoute et accepte des connexions.

Service fingerprinting : Technique de détection de la version exacte d'un service qui tourne sur un port. L'option -sV de Nmap active cette détection — elle permet d'identifier non seulement qu'un service tourne sur le port 3000, mais aussi que c'est Node.js Express.

2.3 Burp Suite

Burp Suite : Suite d'outils professionnelle pour tester la sécurité des applications web. Développée par PortSwigger, elle est utilisée par la grande majorité des pentesters dans le monde. La version Community Edition est gratuite et contient les fonctionnalités essentielles.

Proxy d'interception : Composant de Burp Suite qui se place entre le client et le serveur pour intercepter toutes les communications HTTP. Il permet de lire, modifier et rejouer les requêtes avant qu'elles n'arrivent au serveur.

Repeater : Module de Burp Suite qui permet de modifier et rejouer manuellement des requêtes HTTP. C'est l'outil idéal pour tester des payloads malveillants (SQL Injection, XSS) de manière contrôlée et précise.

Payload : Donnée malveillante envoyée dans une requête pour exploiter une vulnérabilité. Dans notre démonstration, le payload SQL Injection est : ' OR '1'='1.

2.4 OWASP ZAP

OWASP ZAP (Zed Attack Proxy) : Outil open source de test de sécurité développé par OWASP. Il permet de scanner automatiquement une application web pour détecter les

vulnérabilités courantes. C'est l'un des outils de sécurité les plus utilisés dans le monde, entièrement gratuit.

Scan automatisé : Processus où ZAP explore automatiquement toutes les pages et endpoints d'une application, puis lance des tests de sécurité sur chaque point d'entrée trouvé. Il détecte les vulnérabilités sans intervention manuelle.

Araignée web (Spider) : Composant de ZAP qui parcourt automatiquement toutes les URLs d'une application en suivant les liens. Il construit une cartographie complète de l'application avant de lancer les tests de sécurité.

CSP (Content Security Policy) : Header HTTP de sécurité qui définit les sources de contenu autorisées dans une page web. Sans CSP, un attaquant peut plus facilement injecter du contenu malveillant (XSS). ZAP l'a détecté comme manquant dans notre API.

X-Powered-By : Header HTTP qui révèle la technologie utilisée par le serveur. Notre API renvoie 'X-Powered-By: Express', ce qui informe un attaquant qu'on utilise Express.js. En production, on supprime ce header avec `app.disable('x-powered-by')`.

X-Content-Type-Options : Header HTTP de sécurité qui empêche le navigateur d'interpréter les fichiers différemment de leur type MIME déclaré. Sans ce header, un attaquant peut tromper le navigateur pour exécuter du code malveillant.

3. Étapes réalisées

3.1 Scan Nmap — Découverte des ports ouverts

Nmap a été installé et utilisé pour scanner tous les ports ouverts sur le serveur local. Deux commandes ont été exécutées :

Commande 1 — Scan global

```
nmap -sV localhost
```

Cette commande scanne tous les ports et détecte les versions des services. Résultats obtenus :

Port	État	Service	Version détectée
135/tcp	Ouvert	msrpc	Microsoft Windows RPC

445/tcp	Ouvert	microsoft-ds	Partage de fichiers Windows
903/tcp	Ouvert	ssl/vmware-auth	VMware Authentication Daemon 1.10
3000/tcp	Ouvert	http	Node.js Express framework
3306/tcp	Ouvert	mysql	MySQL 8.0.44

Commande 2 — Scan ciblé sur l'API

```
nmap -sV -p 3000 localhost
```

Cette commande cible uniquement le port 3000 de l'API. Résultat :

```
3000/tcp open  http  Node.js Express framework
```

Analyse de sécurité des résultats : Nmap a identifié que le port 3000 expose une API Node.js/Express — un attaquant peut maintenant cibler des vulnérabilités spécifiques à ce framework. Le port 3306 expose MySQL directement, ce qui est dangereux en production. En environnement réel, ces ports seraient filtrés par un firewall.

3.2 Burp Suite — Interception et modification des requêtes

Burp Suite Community Edition a été utilisé pour intercepter et modifier des requêtes HTTP envoyées à l'API. Le module Repeater a été utilisé pour tester les failles depuis l'outil.

Test 1 — Connexion normale

Une requête de connexion légitime a été envoyée depuis le Repeater :

```
POST /api/auth/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"email":"test@gmail.com","password":"monmotdepasse123"}
```

Résultat : 200 OK — Token JWT généré avec succès. Burp Suite a révélé le header X-Powered-By: Express dans la réponse, confirmant la fuite d'information détectée plus tard par OWASP ZAP.

Test 2 — Attaque SQL Injection depuis Burp Suite

La même requête a été modifiée avec un payload SQL Injection :

```
POST /api/vulnerable/login HTTP/1.1
```

```
Host: localhost:3000
Content-Type: application/json

{"email":""," OR '1'='1","password":"test"}
```

Route testée	Payload	Résultat
/api/vulnerable/login	' OR '1'='1	200 — Connexion réussie (VULNÉRABLE)
/api/vulnerable/secure-login	' OR '1'='1	401 — Utilisateur introuvable

3.3 OWASP ZAP — Scan automatique de vulnérabilités

OWASP ZAP a été utilisé pour lancer un scan automatisé sur l'API. L'URL `http://localhost:3000/health` a été utilisée comme point d'entrée. Le scan a duré quelques minutes et a identifié 3 alertes de sécurité.

Alerte	Niveau	Description	Correction recommandée
CSP: Failure to Define Directive with No Fallback	Moyen	Pas de Content Security Policy définie — risque d'injection de contenu malveillant	Ajouter le header Content-Security-Policy dans les réponses
Server Leaks Information via X-Powered-By	Faible	Le header X-Powered-By révèle qu'on utilise Express.js	<code>app.disable('x-powered-by')</code> dans <code>index.ts</code>
X-Content-Type-Options Header Missing	Faible	Header de sécurité manquant — le navigateur peut mal interpréter les fichiers	Ajouter le header X-Content-Type-Options: nosniff

Ces alertes sont typiques d'une API en développement. En production, on les corrigerait en ajoutant un middleware de sécurité comme `Helmet.js` qui configure automatiquement tous ces headers de sécurité.

4. Analyse des résultats

4.1 Ce que les outils ont révélé

Outil	Ce qu'il a trouvé	Impact
-------	-------------------	--------

Nmap	5 ports ouverts dont MySQL exposé	Un attaquant peut cibler directement la base de données
Burp Suite	SQL Injection possible sur /vulnerable/login	Accès aux données sans authentification
OWASP ZAP	3 alertes — CSP, X-Powered-By, X-Content-Type	Fuite d'information et risques XSS

4.2 Corrections recommandées en production

Problème détecté	Correction	Comment
X-Powered-By exposé	Supprimer le header	app.disable('x-powered-by')
CSP manquante	Ajouter Content-Security-Policy	Utiliser le middleware Helmet.js
X-Content-Type-Options manquant	Ajouter le header	helmet() configure ça automatiquement
MySQL exposé (port 3306)	Fermer le port	Configurer un firewall — accès local uniquement
Port 3000 exposé	Utiliser un reverse proxy	Nginx devant Express en production

5. Bilan global du projet

5.1 Récapitulatif des 4 phases

Phase	Contenu	Compétences démontrées
Phase 1 — Initialisation	Node.js + TypeScript + GitHub	Setup professionnel, Git, architecture MVC
Phase 2 — Authentification	JWT + bcrypt + SQLite	Sécurité des mots de passe, tokens, base de données
Phase 3 — Failles	SQL Injection + XSS + JWT none	Offensive ET défensive, OWASP Top 10
Phase 4 — Tests	Nmap + Burp Suite + OWASP ZAP	Outils professionnels de pentesting

5.2 Ce qui a été réalisé

Élément	Statut
---------	--------

Scan Nmap — ports ouverts identifiés	Réalisé
Scan ciblé port 3000 — Express identifié	Réalisé
Burp Suite — connexion normale testée	Réalisé
Burp Suite — SQL Injection démontrée	Réalisé
Burp Suite — route sécurisée testée	Réalisé
OWASP ZAP — scan automatique lancé	Réalisé
OWASP ZAP — 3 alertes identifiées	Réalisé
Corrections recommandées documentées	Réalisé

5.3 Compétences globales démontrées sur l'ensemble du projet

- Développement backend professionnel en Node.js + TypeScript
- Implémentation de mécanismes d'authentification sécurisés (JWT, bcrypt)
- Compréhension et exploitation des failles OWASP Top 10
- Utilisation d'outils professionnels de cybersécurité (Nmap, Burp Suite, OWASP ZAP)
- Pensée offensive ET défensive — comprendre l'attaque pour mieux se défendre
- Documentation professionnelle et utilisation de Git/GitHub
- Capacité à présenter et expliquer des concepts techniques complexes

5.4 Outils utilisés tout au long du projet

Outil	Catégorie	Usage
Node.js + TypeScript	Développement	Construction de l'API REST
Express	Développement	Serveur web et gestion des routes
SQLite + bcrypt + JWT	Sécurité	Base de données et authentification
Postman	Test	Tests manuels des endpoints API
Git + GitHub	Versioning	Gestion du code source professionnel
Nmap	Sécurité	Scan des ports et services ouverts
Burp Suite	Sécurité	Interception et modification des requêtes HTTP
OWASP ZAP	Sécurité	Scan automatique de vulnérabilités